SISTEMA NATYS

Informe de Pruebas de Software

Módulo: Módulo

Integrante	Elias Arma Moises Pereira Edgmairys Aponte Ismael Torres Santiago Loyo
Equipo	Equipo 8
Docente	Profa. Angelismar Terán
Sección	IN3103
Fecha	17-10-2026

1. Introducción

El presente informe documenta el proceso integral de aseguramiento de calidad (QA) y pruebas de software aplicadas a cada uno de los módulos que integran el **Sistema Naty's**. El objetivo principal es verificar y validar el correcto funcionamiento de los flujos de trabajo del sistema, incluyendo la gestión completa del ciclo de vida de los datos (registro, consulta, modificación, eliminación, aprobación y cancelación de acciones).

Para garantizar un nivel óptimo de estabilidad, rendimiento y seguridad, se implementó una estrategia de evaluación multinivel que abarca las siguientes tipologías de pruebas:

- **Pruebas Unitarias y de Humo:** Ejecutadas directamente sobre la capa de modelo mediante **PHPUnit** para verificar el comportamiento aislado de las funciones core y asegurar la estabilidad básica antes de fases avanzadas.
- **Pruebas de Integración y Aceptación:** Realizadas con **PHPUnit** y **Codeception**, validando la interacción entre componentes y garantizando que los módulos cumplan con los criterios de aceptación y requerimientos del usuario.
- **Pruebas de Backend y Seguridad:** Implementadas con **Postman** para auditar el comportamiento de las APIs, el manejo de respuestas HTTP y la resistencia ante vulnerabilidades en los endpoints.
- **Pruebas de Interfaz de Usuario (UI):** Automatizadas con **Selenium** para simular la navegación e interacción directa del usuario en el navegador.
- **Pruebas de Rendimiento y Carga:** Llevadas a cabo mediante **Apache JMeter**, evaluando los tiempos de respuesta y la estabilidad del sistema bajo volúmenes de concurrencia y carga de datos.

Modulo: Módulo

El módulo Módulo se verificó con seis pruebas unitarias (UT-01 a UT-06) que cubren la validación de acciones y datos y el CRUD completo mediante mocks, de modo que se ejecutan sin servidor ni base de datos. Finalmente, una prueba de integración (IT-01) comprobó que el modelo consulta y devuelve módulos reales desde la base de datos de seguridad.

1.1 Resumen de resultados

ID	Caso de prueba	Acción	Estado
UT-01	Acción inválida retorna error de estructura	manejarAccion	**Aprobado**
UT-02	Registrar módulo con datos vacíos	registrar	**Aprobado**
UT-03	Registrar módulo exitosamente con mock	registrar	**Aprobado**
UT-04	Obtener módulo exitosamente con mock	obtener	**Aprobado**

ID	Caso de prueba	Acción	Estado
UT-05	Modificar módulo exitosamente con mock	modificar	**Aprobado**
UT-06	Eliminar módulo exitosamente con mock	eliminar	**Aprobado**
Suite	Archivo completo tests/unitarias/ModuloTest.php (6 pruebas unitarias)	todas	**6 de 6 Aprobado**
IT-01	Integración: consultar módulos en la base de datos real	consultar	**Aprobado**
API-01	Backend: obtener un módulo por su ID (Postman)	obtener	**Aprobado**
SM-01	Humo: el módulo de Módulos inicializa y carga correctamente	todas	**Aprobado**
UI-01	Interfaz y sistema: el formulario Modificar muestra los datos del módulo (Selenium IDE)	modificar	**Aprobado**
SEC-01	Seguridad: obtener un módulo sin sesión (Postman)	obtener	**Aprobado**
SEC-02	Seguridad: consultar la gestión de módulos sin sesión (Postman)	consultar	**Aprobado**
SEC-03	Seguridad: inyección SQL en el identificador del módulo (Postman)	obtener	**Aprobado**
RD-01	Rendimiento: 50 usuarios consultan la gestión de módulos simultáneamente (JMeter)	login / consultar	**Aprobado**
AC-01	Aceptación: consultar el listado de módulos y su gestión (Codeception)	consultar	**Aprobado**

1.2 Cobertura del CRUD

Operación	Prueba que la cubre	Comportamiento verificado
Validación de acciones	UT-01 · Acción inválida	**Rechaza comandos no reconocidos devolviendo status = false y el mensaje «Accion invalida».**
Validación de datos	UT-02 · Registrar módulo con datos vacíos	**El validador detiene el registro cuando el nombre está vacío y devuelve el mensaje «Error: no puede estar vacío los datos».**
Crear / Registrar	UT-03 · Registrar módulo mediante mock	**Inserta el módulo con nombre, descripción, tipo, ruta e ícono válidos y confirma el registro.**
Leer	UT-04 · Obtener módulo mediante mock	**Recupera el módulo y sus datos (id, nombre, descripción, tipo, ruta e ícono) desde la consulta simulada.**
Actualizar	UT-05 · Modificar módulo mediante mock	Actualiza los datos del módulo y confirma la transacción.

Operación	Prueba que la cubre	Comportamiento verificado
Eliminar	UT-06 · Eliminar módulo mediante mock	Desactiva el módulo y sus accesos, y confirma la transacción.

1.3 Prueba unitaria UT-01: Acción inválida retorna error de estructura

Campo	Detalle
ID	UT-01
Nombre	Acción inválida retorna error de estructura
Tipo de prueba	Unitaria
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/unitarias/ModuloTest.php · test_accion_invalida_retorna_error
Descripción	Verifica que el método centralizador manejarAccion rechace cualquier acción no reconocida ('accion_falsa'), devolviendo un arreglo con status = false y el mensaje «Accion invalida» sin afectar la base de datos.
Precondiciones	PHPUnit instalado en el proyecto · Configuración phpunit.xml presente · Entorno aislado sin requerir servidor MySQL activo.
Datos de entrada	**Acción: accion_falsa / JSON: {}**
Pasos	1. Se abrió la terminal CMD en la ruta del proyecto C:\xampp\htdocs\larence. 2. Se verificó PHPUnit con: vendor\bin\phpunit --version 3. Se ejecutó la prueba unitaria específica: vendor\bin\phpunit tests\unitarias\ ModuloTest.php --filter test_accion_invalida_retorna_error
Resultado esperado	El modelo procesa la acción desconocida, devuelve un arreglo con status = false y el mensaje de error «Accion invalida».
Resultado obtenido	El modelo devolvió correctamente status = false y el mensaje esperado. PHPUnit validó satisfactoriamente todas las aserciones de la estructura de error.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

1.3.1 Evidencias

Captura 1 – Código del método aislado

```php
/** @test */
public function test_accion_invalida_retorna_error(): void {
    $modulo = new Modulo();
    $resultado = $modulo->manejarAccion('accion_falsa', '{}');

    $this->assertIsArray($resultado);
    $this->assertFalse($resultado['status']);
    $this->assertEquals('Accion invalida', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-01

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\MonedaTest.php --filter test_accion_invalida_retorna_error
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.055, Memory: 8.00 MB

OK, but there were issues!
Tests: 1, Assertions: 3, PHPUnit Deprecations: 6.
```

Prueba aprobada con XAMPP apagado

2.1 Prueba unitaria UT-02: Registrar módulo con datos vacíos

ID	UT-02
Nombre	Registrar módulo con datos vacíos
Tipo de prueba	Unitaria
Acción evaluada	registrar
Archivo y método	tests/unitarias/ModuloTest.php · test_registrar_modulo_con_datos_vacios
Descripción	Verifica que el método registrar rechace una solicitud con el nombre vacío, deteniendo el proceso antes de insertar el módulo y devolviendo status = false.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · PHPUnit configurado y operativo en el proyecto.
Simulación (mock)	No requiere simulación de PDO: el validador de datos detiene la operación antes de cualquier acceso a la base de datos.
Datos de entrada	**`Acción: registrar / JSON: {"nombre": ""}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/ModuloTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_registrar_modulo_con_datos_vacios --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El validador detecta el nombre vacío, no registra el módulo y devuelve status = false con el mensaje «Error: no puede estar vacío los datos».
Resultado obtenido	✔️ Registrar modulo con datos vacios. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo rechazó la solicitud incompleta sin acceder a la base de datos.

Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_registrar_modulo_con_datos_vacios(): void {
    $modulo = new Modulo();
    $resultado = $modulo->manejarAccion('registrar', json_encode(['nombre' => '']));

    $this->assertIsArray($resultado);
    $this->assertFalse($resultado['status']);
    $this->assertEquals('Error: no puede estar vacío los datos', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-02

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_accion_invalida_retorna_error
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.056, Memory: 8.00 MB

OK, but there were issues!
Tests: 1, Assertions: 3, PHPUnit Deprecations: 6.
```

Prueba aprobada con XAMPP apagado

2.2 Prueba unitaria UT-03: Registrar módulo exitosamente con mock

ID	UT-03
Nombre	Registrar módulo exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	registrar
Archivo y método	tests/unitarias/ModuloTest.php · test_registrar_modulo_exitosamente_con_mock
Descripción	Verifica que el método registrar inserte un módulo con nombre, descripción, tipo, ruta e ícono válidos mediante una conexión PDO simulada, retornando status = true y el mensaje «Modulo registrado con exito.».
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · ModuloTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO y la sentencia PDOStatement para interceptar prepare y execute, confirmando la ejecución del INSERT sobre la tabla modulos.
Datos de entrada	**`Acción: registrar / JSON: {"id": 1, "nombre": "Ventas", "descripcion": "Módulo de ventas", "tipoModulo": "Principal", "ruta": "ventas", "icono": "cart"}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/ModuloTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_registrar_modulo_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.

Resultado esperado	El modelo valida los datos, registra el módulo y devuelve status = true con el mensaje «Modulo registrado con exito.».
Resultado obtenido	✔️ Registrar modulo exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo registró el módulo y confirmó la operación simulada.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_registrar_modulo_exitosamente_con_mock(): void {
    $stmtMock = $this->createMock(PDOStatement::class);
    $stmtMock->expects($this->once())->method('execute')->willReturn(true);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->expects($this->once())->method('prepare')->willReturn($stmtMock);

    $moduloMock = $this->getMockBuilder(Modulo::class)
                       ->onlyMethods(['getConnectionSeguridad', 'closeConnection'])
                       ->getMock();

    $moduloMock->method('getConnectionSeguridad')->willReturn($pdoMock);
    $moduloMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([
        'id'          => 1,
        'nombre'      => 'Ventas',
        'descripcion' => 'Módulo de ventas',
        'tipoModulo'  => 'Principal',
        'ruta'        => 'ventas',
        'icono'       => 'cart'
    ]);

    $resultado = $moduloMock->manejarAccion('registrar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
    $this->assertEquals('Modulo registrado con exito.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-03

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_registrar_modulo_exitosamente_con_mock
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.111, Memory: 10.00 MB

OK, but there were issues!
Tests: 1, Assertions: 5, PHPUnit Deprecations: 6.
```

Prueba aprobada con XAMPP apagado

2.3 Prueba unitaria UT-04: Obtener módulo exitosamente con mock

ID	UT-04

Nombre	Obtener módulo exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	obtener
Archivo y método	tests/unitarias/ModuloTest.php · test_obtener_modulo_exitosamente_con_mock
Descripción	Verifica que el método obtener recupere un módulo existente mediante una consulta PDO simulada, retornando status = true, el mensaje «Modulo encontrado.» y los datos del módulo (id_modulo, nombre, descripción, tipo, ruta e ícono).
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · ModuloTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO y la sentencia PDOStatement para interceptar bindValue(':id', 1), execute y fetch, devolviendo un módulo con id_modulo = 1, nombre_modulo = «Inventario», descripcion = «Control de stock», tipo_modulo = «Principal», ruta = «inventario» e icono = «box».
Datos de entrada	**`Acción: obtener / JSON: {"id": 1, "nombre": "Inventario", "descripcion": "Control de stock", "tipoModulo": "Principal", "ruta": "inventario", "icono": "box"}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/ModuloTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_obtener_modulo_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo devuelve status = true, el mensaje «Modulo encontrado.» y los datos del módulo consultado (id_modulo = 1) sin requerir conexión física a la base de datos.
Resultado obtenido	✔️ Obtener modulo exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo devolvió los datos esperados mediante el aislamiento con mocks.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_obtener_modulo_exitosamente_con_mock(): void {
    $stmtMock = $this->createMock(PDOStatement::class);
    $stmtMock->expects($this->once())->method('bindValue')->with($this->equalTo(':id'), $this->equalTo(1));
    $stmtMock->expects($this->once())->method('execute')->willReturn(true);
    $stmtMock->expects($this->once())->method('fetch')->willReturn([
        'id_modulo'     => 1,
        'nombre_modulo' => 'Inventario',
        'descripcion'   => 'Control de stock',
        'tipo_modulo'   => 'Principal',
        'ruta'          => 'inventario',
        'icono'         => 'box'
    ]);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->expects($this->once())->method('prepare')->willReturn($stmtMock);

    $moduloMock = $this->getMockBuilder(Modulo::class)
                       ->onlyMethods(['getConnectionSeguridad', 'closeConnection'])
                       ->getMock();

    $moduloMock->method('getConnectionSeguridad')->willReturn($pdoMock);
    $moduloMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([
        'id'          => 1,
        'nombre'      => 'Inventario',
        'descripcion' => 'Control de stock',
        'tipoModulo'  => 'Principal',
        'ruta'        => 'inventario',
        'icono'       => 'box'
    ]);

    $resultado = $moduloMock->manejarAccion('obtener', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
    $this->assertEquals('Modulo encontrado.', $resultado['msj']);
    $this->assertEquals(1, $resultado['data']['id_modulo']);
}
```

Captura 2 – Ejecución de UT-04

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_obtener_modulo_exitosamente_con_mock
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.096, Memory: 10.00 MB

OK, but there were issues!
Tests: 1, Assertions: 8, PHPUnit Deprecations: 6.
```

Prueba aprobada con XAMPP apagado

2.4 Prueba unitaria UT-05: Modificar módulo exitosamente con mock

ID	UT-05
Nombre	Modificar módulo exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	modificar
Archivo y método	tests/unitarias/ModuloTest.php · test_modificar_modulo_exitosamente_con_mock

Descripción	Verifica que el método modificar actualice los datos de un módulo existente y confirme la transacción, devolviendo status = true y el mensaje «Modulo actualizado con exito.» sin requerir base de datos física.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · ModuloTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO y las sentencias PDOStatement para interceptar la consulta del módulo (SELECT) y la actualización (UPDATE modulos), confirmando la ejecución de la transacción.
Datos de entrada	**`Acción: modificar / JSON: {"id": 1, "nombre": "Inventario V2", "descripcion": "Control de stock v2", "tipoModulo": "Principal", "ruta": "inventario", "icono": "box"}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/ModuloTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_modificar_modulo_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo procesa la actualización, confirma la transacción y devuelve status = true con el mensaje «Modulo actualizado con exito.».
Resultado obtenido	✔️ Modificar modulo exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo actualizó el módulo y confirmó la transacción simulada.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_modificar_modulo_exitosamente_con_mock(): void {
    $stmtSelect = $this->createMock(PDOStatement::class);
    $stmtSelect->method('execute')->willReturn(true);
    $stmtSelect->method('fetch')->willReturn(['id_modulo' => 1, 'nombre_modulo' => 'Inventario']);

    $stmtUpdate = $this->createMock(PDOStatement::class);
    $stmtUpdate->method('execute')->willReturn(true);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtSelect, $stmtUpdate) {
        if (strpos($sql, 'UPDATE modulos') !== false) return $stmtUpdate;
        return $stmtSelect;
    });

    $moduloMock = $this->getMockBuilder(Modulo::class)
                       ->onlyMethods(['getConnectionSeguridad', 'closeConnection'])
                       ->getMock();

    $moduloMock->method('getConnectionSeguridad')->willReturn($pdoMock);
    $moduloMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([
        'id'          => 1,
        'nombre'      => 'Inventario V2',
        'descripcion' => 'Control de stock v2',
        'tipoModulo'  => 'Principal',
        'ruta'        => 'inventario',
        'icono'       => 'box'
    ]);

    $resultado = $moduloMock->manejarAccion('modificar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
    $this->assertEquals('Modulo actualizado con exito.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-05

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_modificar_modulo_exitosamente_con_mock
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.085, Memory: 10.00 MB

OK, but there were issues!
Tests: 1, Assertions: 3, PHPUnit Deprecations: 6.
```

Prueba aprobada con XAMPP apagado

2.5 Prueba unitaria UT-06: Eliminar módulo exitosamente con mock

ID	UT-06
Nombre	Eliminar módulo exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	eliminar
Archivo y método	tests/unitarias/ModuloTest.php · test_eliminar_modulo_exitosamente_con_mock
Descripción	Verifica que al ejecutar la acción eliminar con un módulo válido, el modelo lo desactive junto con sus accesos y confirme la transacción, devolviendo status = true y el mensaje «Modulo eliminado con exito.».
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · ModuloTest.php configurado con mocks de PDO y PDOStatement.

Simulación (mock)	Se simuló la conexión PDO para interceptar la consulta del módulo (SELECT), la desactivación (UPDATE modulos) y la actualización de accesos (UPDATE accesos), confirmando la transacción.
Datos de entrada	**Acción: eliminar / JSON: {"id": 1, "nombre": "Inventario", "descripcion": "Control de stock", "tipoModulo": "Principal", "ruta": "inventario", "icono": "box"}**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/ModuloTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\ModuloTest.php --filter test_eliminar_modulo_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo desactiva el módulo y sus accesos, confirma la transacción y devuelve status = true con el mensaje «Modulo eliminado con exito.».
Resultado obtenido	✔️ Eliminar modulo exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo confirmó la transacción simulada y desactivó el módulo correctamente.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_eliminar_modulo_exitosamente_con_mock(): void {
    $stmtSelect = $this->createMock(PDOStatement::class);
    $stmtSelect->method('execute')->willReturn(true);
    $stmtSelect->method('fetch')->willReturn(['id_modulo' => 1, 'nombre_modulo' => 'Inventario']);

    $stmtDelete = $this->createMock(PDOStatement::class);
    $stmtDelete->method('execute')->willReturn(true);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtSelect, $stmtDelete) {
        if (strpos($sql, 'UPDATE modulos') !== false || strpos($sql, 'UPDATE accesos') !== false) return $stmtDelete;
        return $stmtSelect;
    });

    $moduloMock = $this->getMockBuilder(Modulo::class)
                       ->onlyMethods(['getConnectionSeguridad', 'closeConnection'])
                       ->getMock();

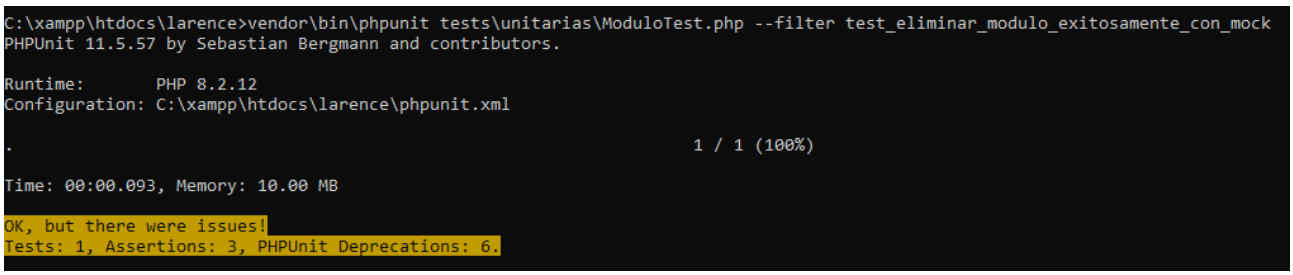
    $moduloMock->method('getConnectionSeguridad')->willReturn($pdoMock);
    $moduloMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([
        'id'          => 1,
        'nombre'      => 'Inventario',
        'descripcion' => 'Control de stock',
        'tipoModulo'  => 'Principal',
        'ruta'        => 'inventario',
        'icono'       => 'box'
    ]);

    $resultado = $moduloMock->manejarAccion('eliminar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
    $this->assertEquals('Modulo eliminado con exito.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-06

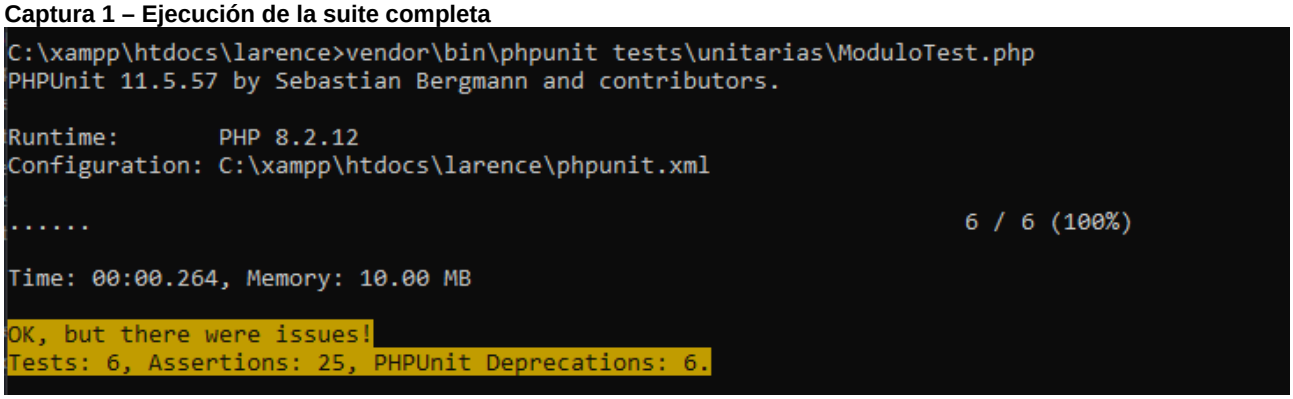


Prueba aprobada con XAMPP apagado

2.6 Ejecución completa de la suite

Comando	vendor\bin\phpunit tests\unitarias\ModuloTest.php --testdox
Precondiciones	XAMPP apagado (Apache y MySQL detenidos).
Resultado esperado	Las 6 pruebas del modelo Módulo aprobadas sin conexión a la base de datos.
Resultado obtenido	6 de 6 pruebas aprobadas, 25 aserciones, sin fallos (0,276 s; 10 MB). PHPUnit mostró 6 avisos «PHPUnit Deprecations» por el uso de la anotación /** @test */ en comentarios, un estilo que PHPUnit 11 marca como obsoleto. No afectan los resultados.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Ejecución de la suite completa



6 pruebas aprobadas con XAMPP apagado

3.1 Prueba de integración IT-01

A diferencia de las pruebas unitarias, la prueba de integración usa el modelo Módulo real conectado a la base de datos de seguridad, para comprobar que el código y la base de datos trabajan correctamente en conjunto y que el listado de módulos coincide con los registros persistidos.

ID	IT-01
Nombre	Integración: consultar módulos en la base de datos real
Tipo de prueba	Integración (Modelo Módulo + MySQL)
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/integracion/ModuloIntegracionTest.php · test_consultar_modulos_en_la_base_real
Descripción	Instancia el modelo Módulo real, ejecuta la acción consultar y compara el primer registro devuelto con la tabla modulos (status = 1) en la base de datos de seguridad, comprobando que la respuesta del modelo coincide con la base de datos.

Precondiciones	MySQL activo en XAMPP · Base de datos de seguridad con al menos un módulo activo.
Datos de entrada	**`Acción: consultar / JSON: {}`**
Pasos	1. Se encendió y verificó el servicio MySQL en el panel de control de XAMPP. 2. Se creó y configuró el archivo tests/integracion/ModuloIntegracionTest.php. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\integracion\ ModuloIntegracionTest.php --testdox 4. Se registró la salida exitosa obtenida en consola.
Resultado esperado	El modelo devuelve status = true con el listado de módulos y el primer id_modulo coincide con un registro activo real de la tabla modulos; si no hay registros, devuelve el mensaje «No se encontraron modulos.».
Resultado obtenido	1 / 1 prueba ejecutada con éxito. OK (1 test, 3 aserciones). 0,308 s; 8 MB. El modelo respondió correctamente y la consulta directa a MySQL confirmó que el módulo devuelto existe en la base de datos real.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código de la prueba de integración

```php
public function test_consultar_modulos_en_la_base_real(): void {
    $moduloModelo = new Modulo();
    $resultado = $moduloModelo->manejarAccion('consultar', '{}');

    $this->assertIsArray($resultado);
    if ($resultado['status']) {
        $this->assertNotEmpty($resultado['data']);
        $idModulo = (int)$resultado['data'][0]['id_modulo'];
        $moduloDb = $this->db->query("SELECT id_modulo FROM modulos WHERE id_modulo = $idModulo AND status = 1")->fetch(PDO::FETCH_ASSOC);
        $this->assertNotFalse($moduloDb);
    } else {
        $this->assertEquals('No se encontraron modulos.', $resultado['msj']);
    }
}
```

Captura 2 – Resultado obtenido de IT-01

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\integracion\ModuloIntegracionTest.php
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.082, Memory: 8.00 MB

OK (1 test, 3 assertions)
```

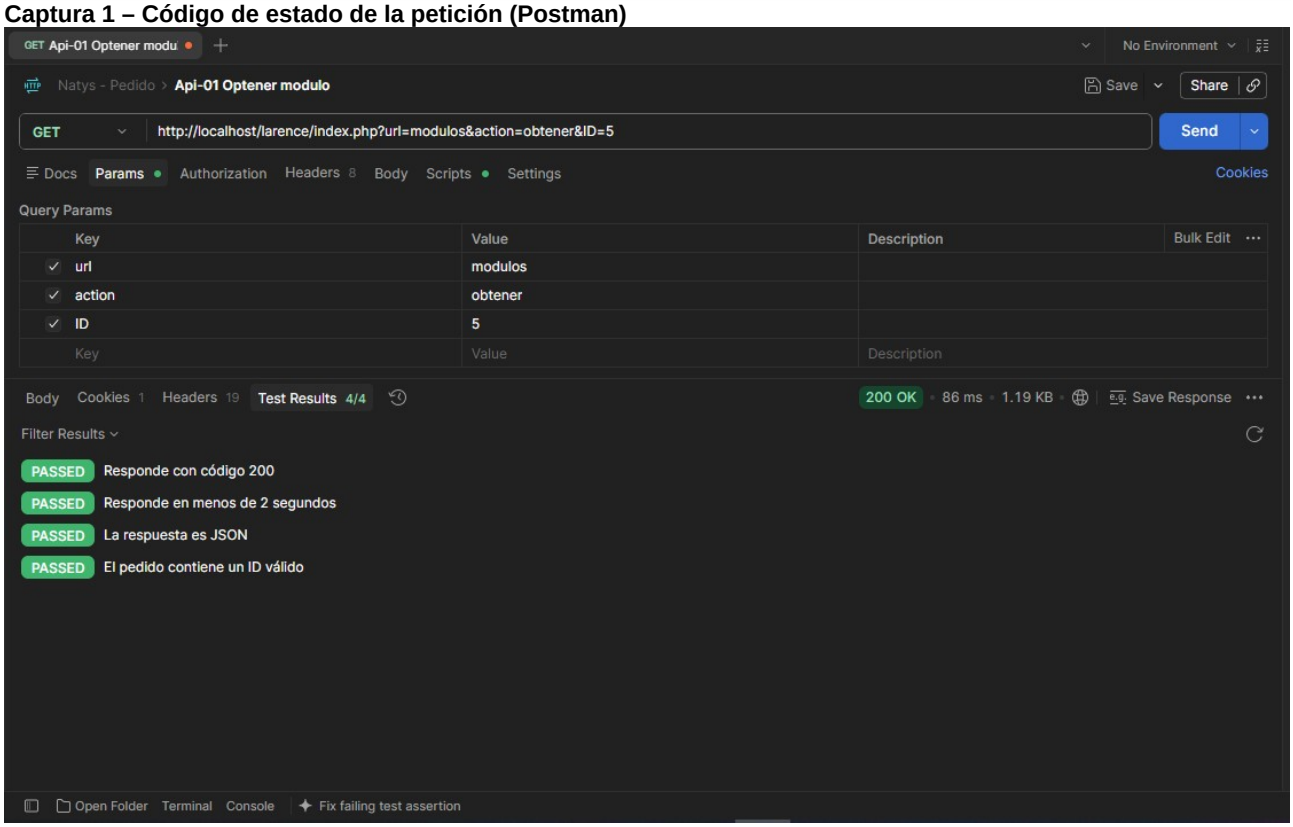
Resultado de la prueba de integración

4.1 Prueba de backend (API) API-01: obtener un módulo por su ID

La prueba de backend se comunica directamente con el controlador mediante peticiones HTTP, sin usar la interfaz web, para verificar que el endpoint del módulo Módulo devuelva el código de estado y el JSON esperados.

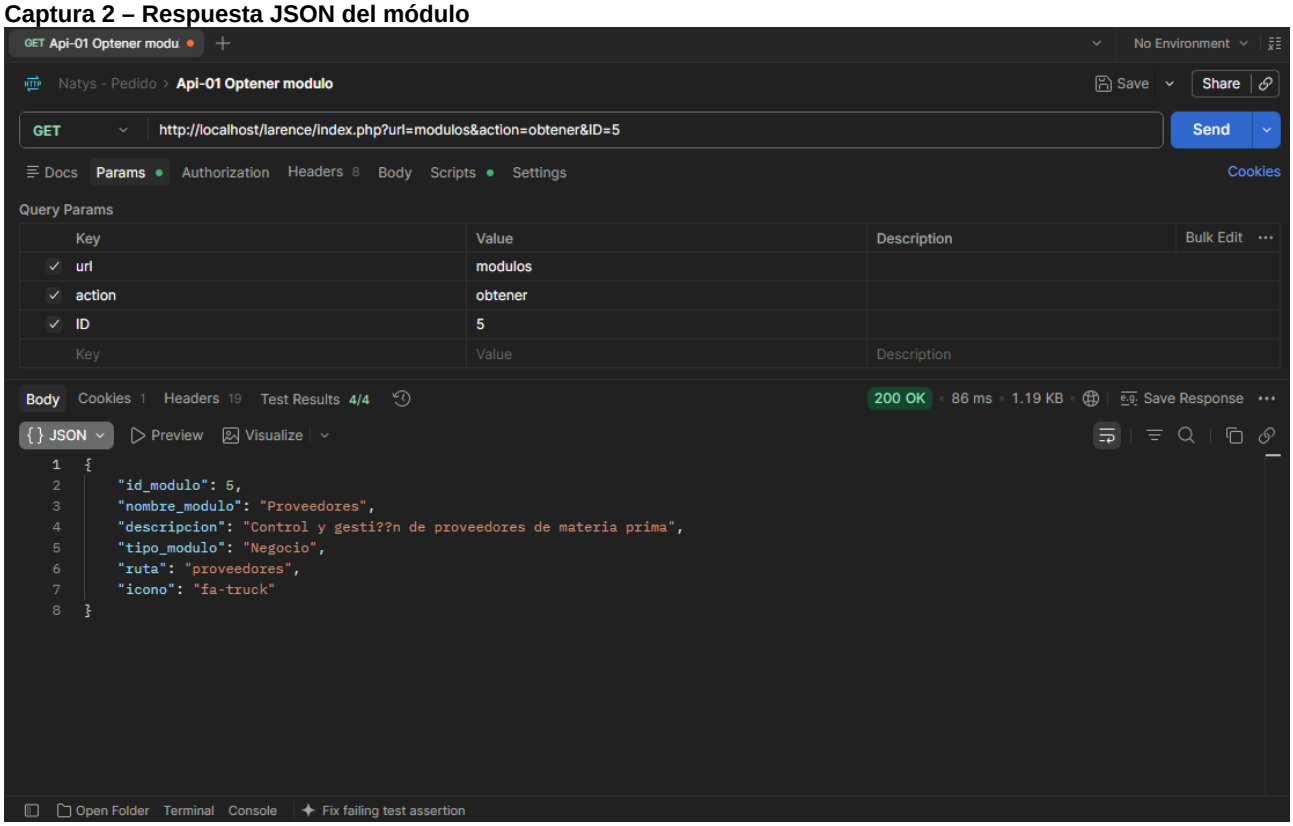
ID	API-01
Nombre	Obtener un módulo por su ID
Tipo de prueba	Backend (API)
Herramienta	Postman · colección «Natys – Módulo»
Petición	`GET http://localhost/larence/index.php?url=modulos&action=obtener&ID=1`
Descripción	Verifica que el endpoint obtener del ModuloController responda con los datos del módulo solicitado en formato JSON.
Precondiciones	Apache y MySQL activos en XAMPP · sesión iniciada en Natys · cookie de sesión PHPSESSID copiada desde el navegador.
Datos de entrada	`Encabezado: Cookie: PHPSESSID=<sesión activa>` `Parámetro: ID=1`
Pasos	1. Se inició sesión en Natys y se copió la cookie PHPSESSID desde las herramientas del navegador (Application → Cookies). 2. En Postman se creó la colección «Natys – Módulo» y la petición GET «API-01 Obtener módulo». 3. Se agregó el encabezado Cookie y el parámetro ID=1. 4. Se envió la petición (Send) y se verificó el código de estado. 5. Se revisó la respuesta JSON en la pestaña Body (Pretty).
Resultado esperado	Código 200, respuesta JSON en menos de 2 segundos, con los datos del módulo 1: nombre, descripción, tipo, ruta, icono y estado.
Resultado obtenido	Código 200 OK; respuesta JSON válida con los datos del módulo 1. La consola de Postman muestra una única petición GET con estado 200, sin redirecciones.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código de estado de la petición (Postman)



200 OK en la petición del módulo

Captura 2 – Respuesta JSON del módulo



JSON con los datos del módulo solicitado

5.1 Prueba de humo SM-01: Módulo

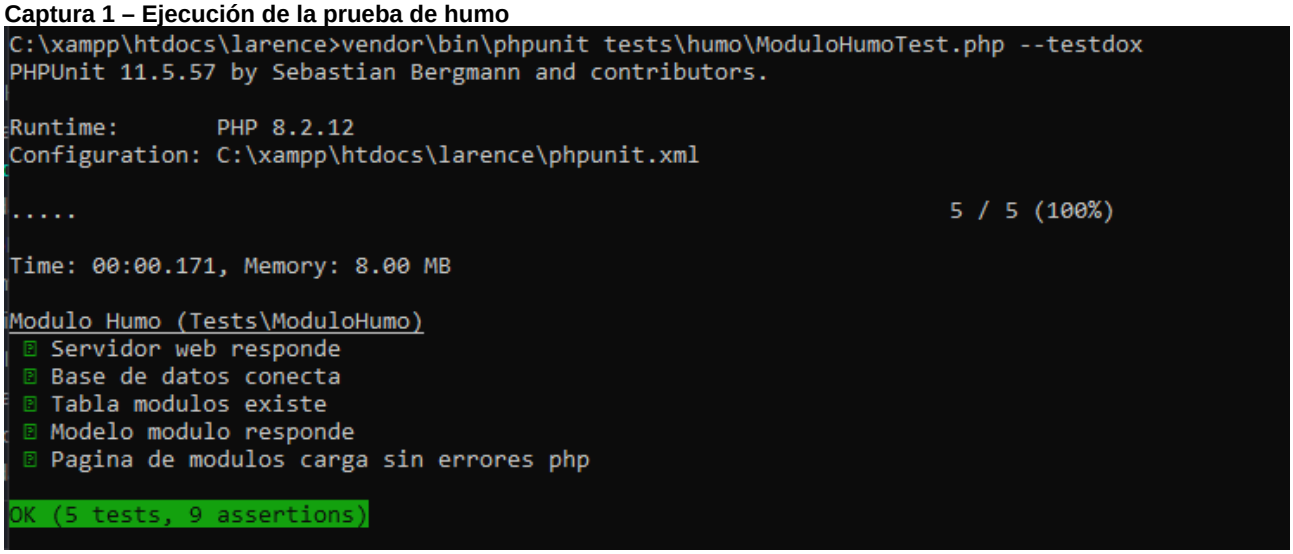
La prueba de humo es una verificación rápida que se ejecuta antes de las pruebas largas para confirmar que el módulo enciende correctamente: el servidor web, la base de datos, la tabla, el modelo y la página responden sin errores.

ID	SM-01
Nombre	El módulo de Módulos enciende correctamente
Tipo de prueba	Humo (smoke test)
Herramienta	PHPUnit 11.5.57
Archivo	tests/humo/ModuloHumoTest.php (5 verificaciones)
Descripción	Comprueba en menos de un segundo que el servidor web responde, que la base de datos conecta, que existe la tabla modulos, que el modelo Modulo responde y que la página del módulo carga sin errores de PHP.
Precondiciones	Apache y MySQL encendidos en XAMPP.
Pasos	1. Se encendieron Apache y MySQL en XAMPP. 2. Se creó el archivo tests/humo/ModuloHumoTest.php. 3. Se ejecutó: vendor\bin\phpunit tests/humo/ModuloHumoTest.php --testdox 4. Se registró la salida de la terminal.
Resultado esperado	Las 5 verificaciones aprobadas en pocos milisegundos.
Resultado obtenido	OK: 5 pruebas y 9 aserciones aprobadas en menos de un segundo.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

#	Verificación	Si fallara, indicaría…	Resultado

1	El servidor web responde (código 200)	Apache apagado	**Aprobado**
2	La base de datos conecta (SELECT 1)	MySQL apagado o mal configurado	**Aprobado**
3	Existe la tabla modulos	Base de datos incompleta	**Aprobado**
4	El modelo Modulo responde a consultar	Error grave en el código del modelo	**Aprobado**
5	La página de Módulos carga sin errores de PHP	Fatal error, Parse error o excepción no capturada	**Aprobado**

Captura 1 – Ejecución de la prueba de humo



5 verificaciones aprobadas, 9 aserciones

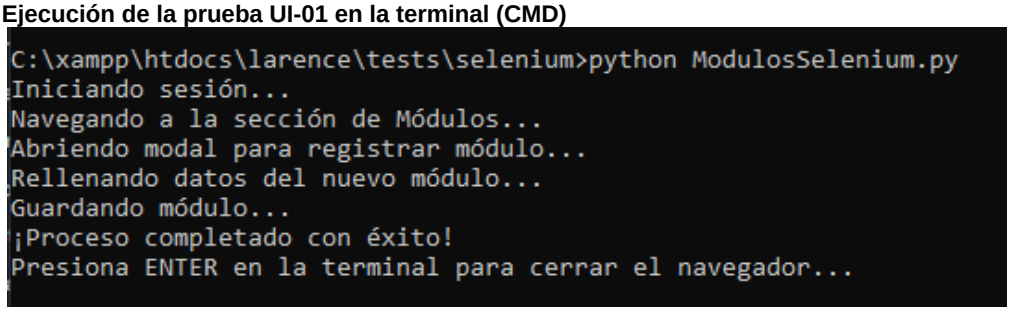
6.1 Prueba de interfaz y sistema UI-01: Módulo

La prueba de interfaz se ejecuta desde el navegador, como lo haría un usuario real: un robot abre la pantalla de Módulo, presiona los botones y verifica lo que aparece. Al recorrer el módulo sin mirar el código, también funciona como prueba de sistema (caja negra).

ID	UI-01
Nombre	El formulario Modificar muestra los datos del módulo
Tipo de prueba	Interfaz y sistema / funcional (caja negra)
Herramienta	Selenium IDE (extensión de Firefox)
Descripción	Abre el módulo Seguridad, presiona Modificar en el primer módulo y verifica que el formulario se abra con el nombre, el tipo y la ruta cargados.
Precondiciones	Apache y MySQL activos · sesión iniciada en Natys en Firefox · existe al menos un módulo registrado
Pasos	1. Se instaló Selenium IDE en Firefox y se creó el proyecto «Natys Módulos». 2. Se abrió index.php?url=modulos con la sesión iniciada. 3. Se presionó Modificar en el primer módulo de la tabla. 4. Se verificaron el título del formulario y los campos nombre, tipo y ruta.
Resultado esperado	El robot abre el formulario «Modificar Modulo» con el nombre, el tipo y la ruta del módulo seleccionado.
Resultado obtenido	Los 8 pasos se ejecutaron en verde y la terminal muestra la prueba aprobada.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

#	Comando	Objetivo	Qué hace
1	open	index.php?url=modulos	**Abre el módulo Seguridad**
2	set window size	1888x1096	**Fija el tamaño de la ventana**
3	wait for element visible	css=#add-row tbody tr.odd	**Espera a que la tabla muestre los módulos**
4	click	css=#add-row tbody tr.odd:nth-child(1) .btn:nth-child(2)	**Presiona Modificar en el primer módulo**
5	pause	2000	**Espera la carga de los datos**
6	assert text	id=moduloModalModificarLabel = «Modificar Modulo»	**Verifica el título del formulario**
7	assert element present	id=nombreModuloEdit	**Verifica el campo del nombre**
8	assert element present	id=tipoModuloEdit	**Verifica el campo del tipo**

Captura 1 – Ejecución de la prueba UI-01 en la terminal (CMD)



La terminal muestra la prueba ejecutada y aprobada

Captura 2 – Vista del módulo durante la prueba UI-01



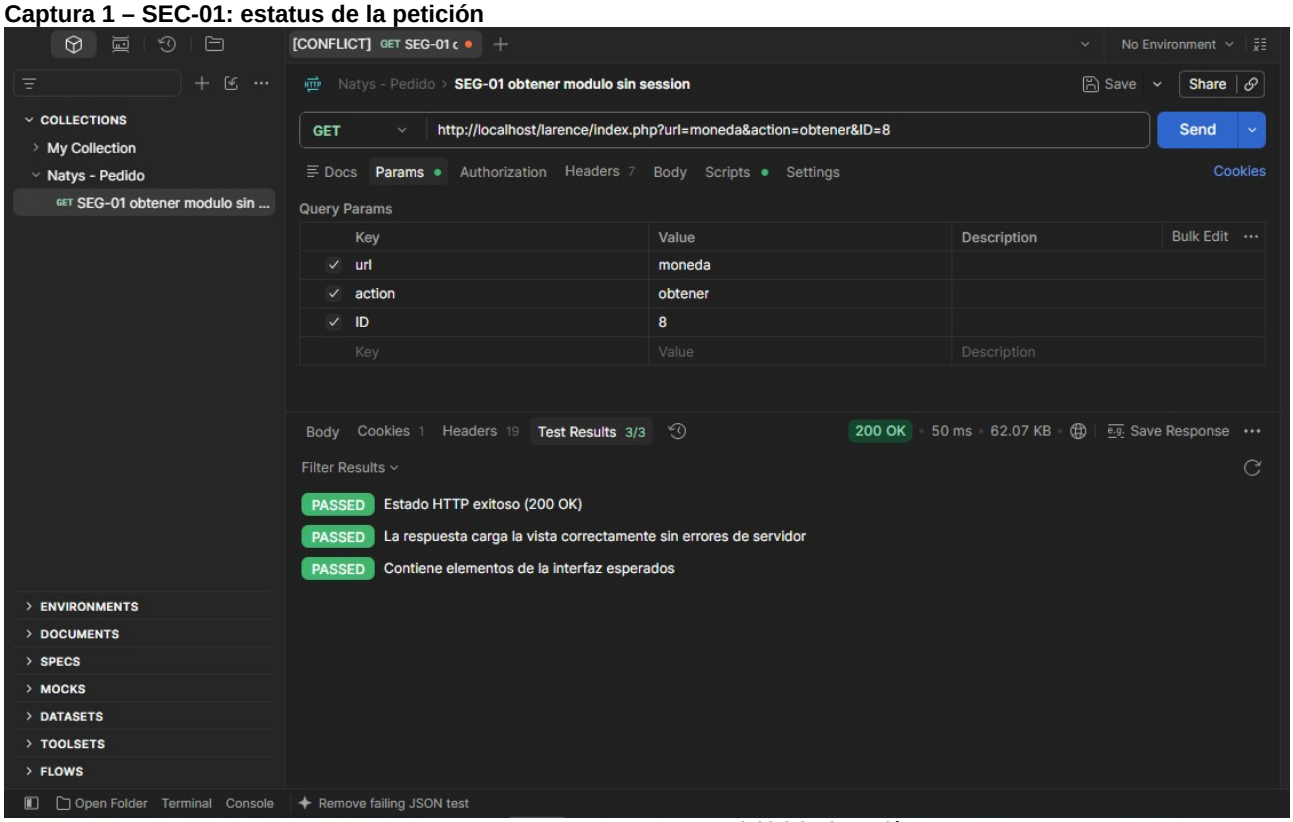
Vista «Seguridad» con la tabla de módulos registrados

7.1 Prueba de seguridad SEC-01: obtener un módulo sin sesión

La prueba intenta obtener un registro del módulo sin haber iniciado sesión, para comprobar que el sistema no expone datos a usuarios anónimos.

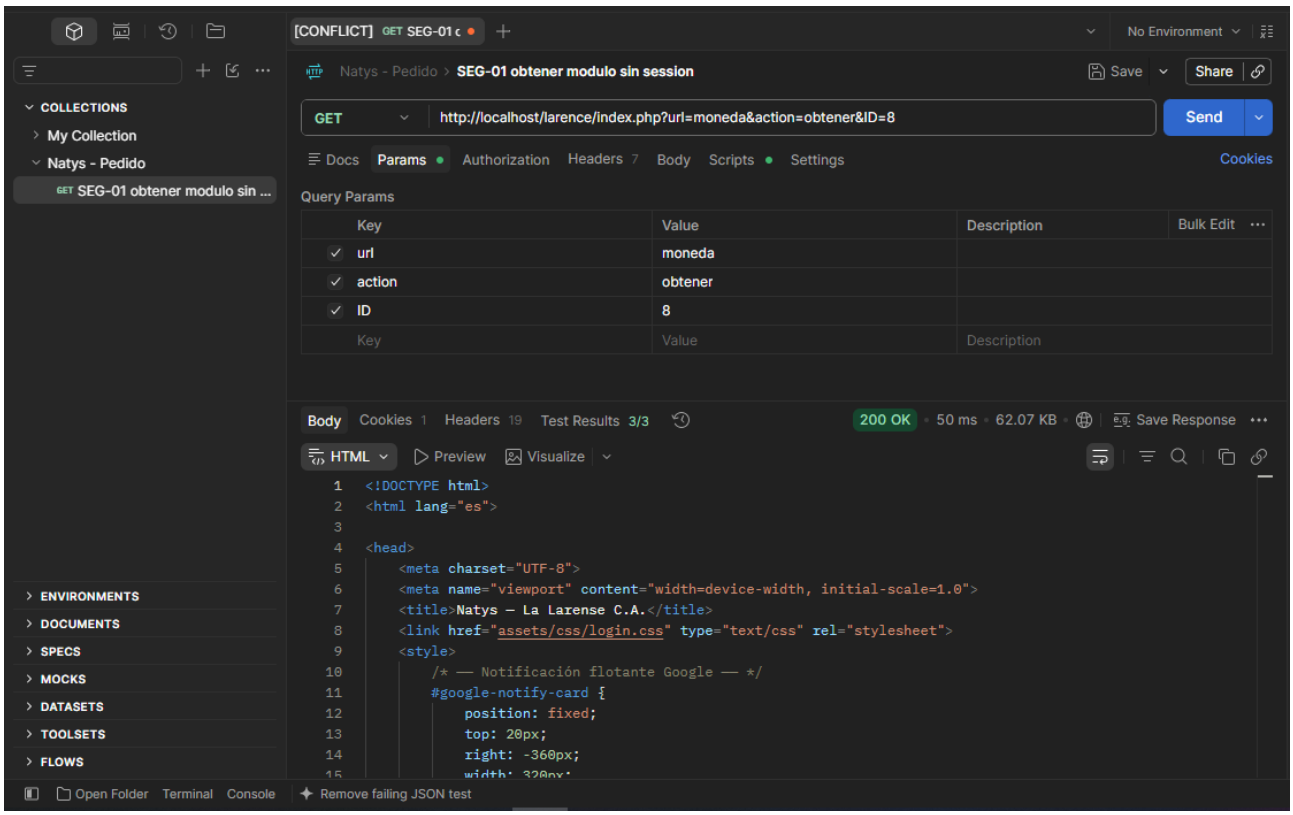
ID	SEC-01
Nombre	Un usuario sin sesión no puede obtener un módulo
Tipo de prueba	Seguridad (control de acceso)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=modulos&action=obtener&ID=8`
Precondiciones	Apache y MySQL activos · petición sin una sesión autenticada · redirecciones seguidas por Postman
Pasos	1. Se duplicó la petición API-01 y se renombró como SEC-01. 2. Se eliminó la cookie de autenticación de la petición. 3. Se envió la petición GET y se siguió la redirección. 4. Se registraron el código final y la respuesta HTML del servidor.
Resultado esperado	El servidor redirige al inicio de sesión y la respuesta final no incluye datos del módulo.
Resultado obtenido	200 OK en 50 ms tras seguir la redirección. La respuesta final es la página de inicio de sesión y no contiene datos del módulo. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-01: estatus de la petición



200 OK en 50 ms y respuesta HTML del inicio de sesión

Captura 2 – SEC-01: respuesta del servidor (HTML)



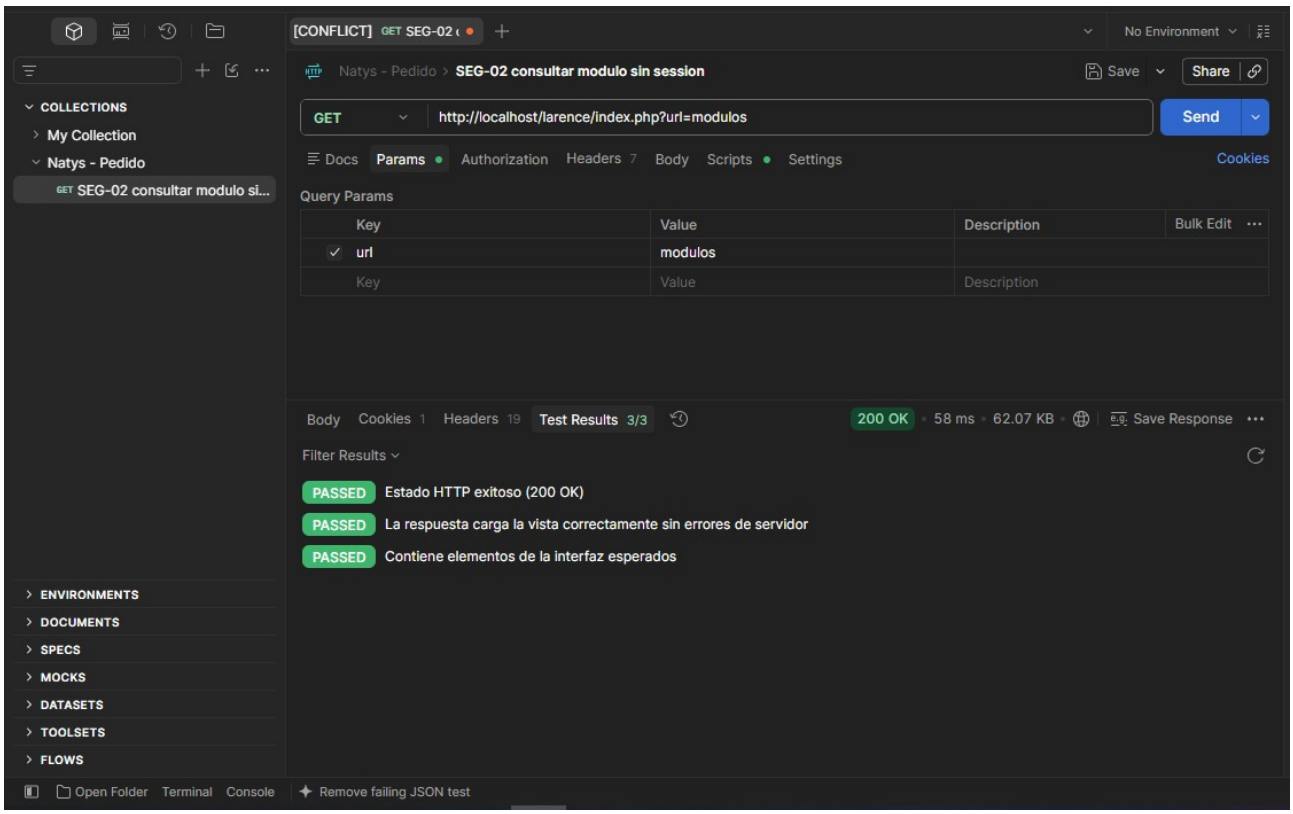
La respuesta final no contiene datos del módulo

7.2 Prueba de seguridad SEC-02: consultar la gestión de módulos sin sesión

La prueba intenta consultar el módulo sin haber iniciado sesión, para comprobar que la página y sus datos quedan protegidos detrás del inicio de sesión.

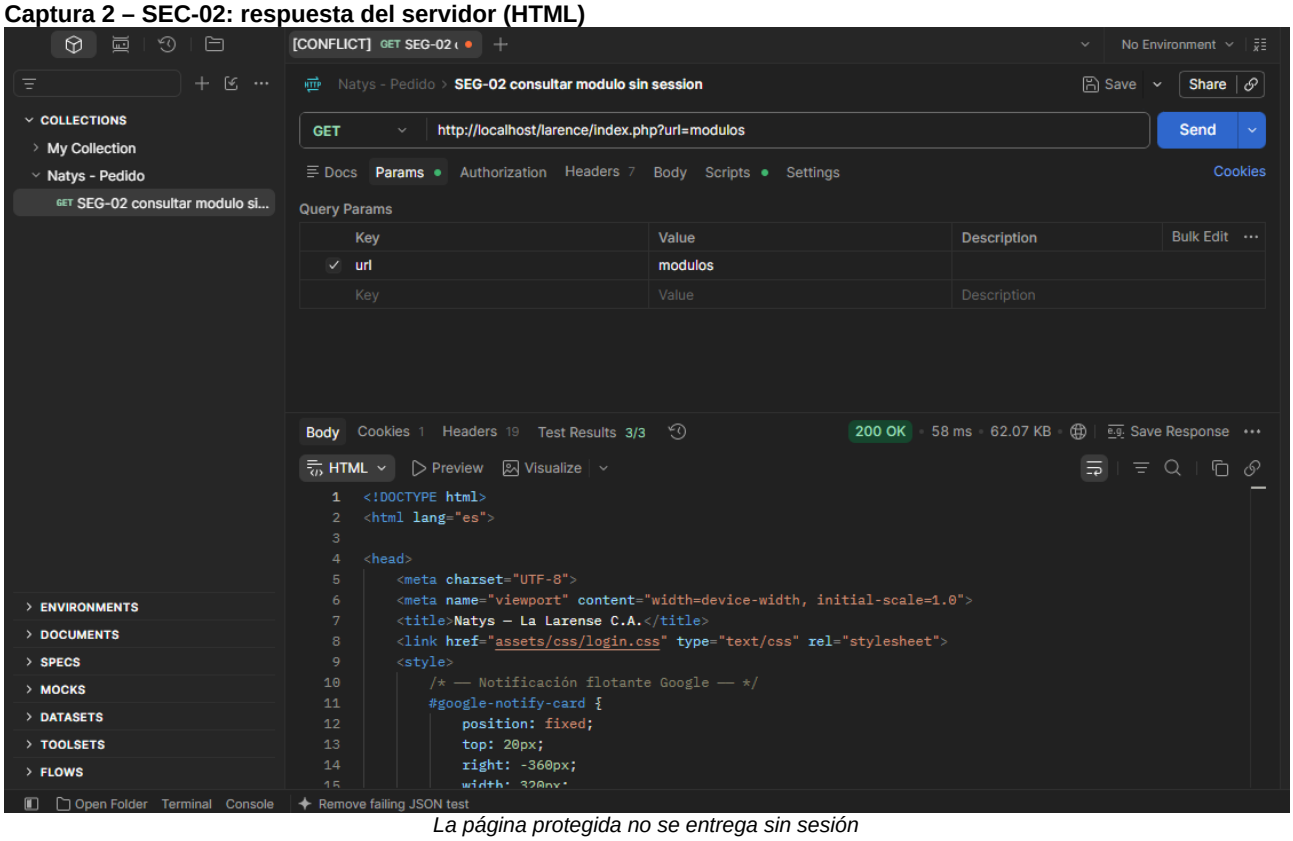
ID	SEC-02
Nombre	Un usuario sin sesión no puede abrir la gestión de módulos
Tipo de prueba	Seguridad (control de acceso)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=modulos`
Precondiciones	Apache y MySQL activos · petición sin una sesión autenticada · redirecciones seguidas por Postman
Pasos	1. Se duplicó la petición de consulta del módulo y se renombró como SEC-02. 2. Se eliminó la cookie de autenticación de la petición. 3. Se envió la petición GET y se siguió la redirección. 4. Se registraron el código final y la respuesta HTML del servidor.
Resultado esperado	El servidor redirige al inicio de sesión y la respuesta final no entrega la página del módulo.
Resultado obtenido	200 OK en 58 ms tras seguir la redirección. La respuesta final es la página de inicio de sesión y no entrega la página del módulo. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-02: estatus de la petición



200 OK en 58 ms y redirección seguida al inicio de sesión

Captura 2 – SEC-02: respuesta del servidor (HTML)



La página protegida no se entrega sin sesión

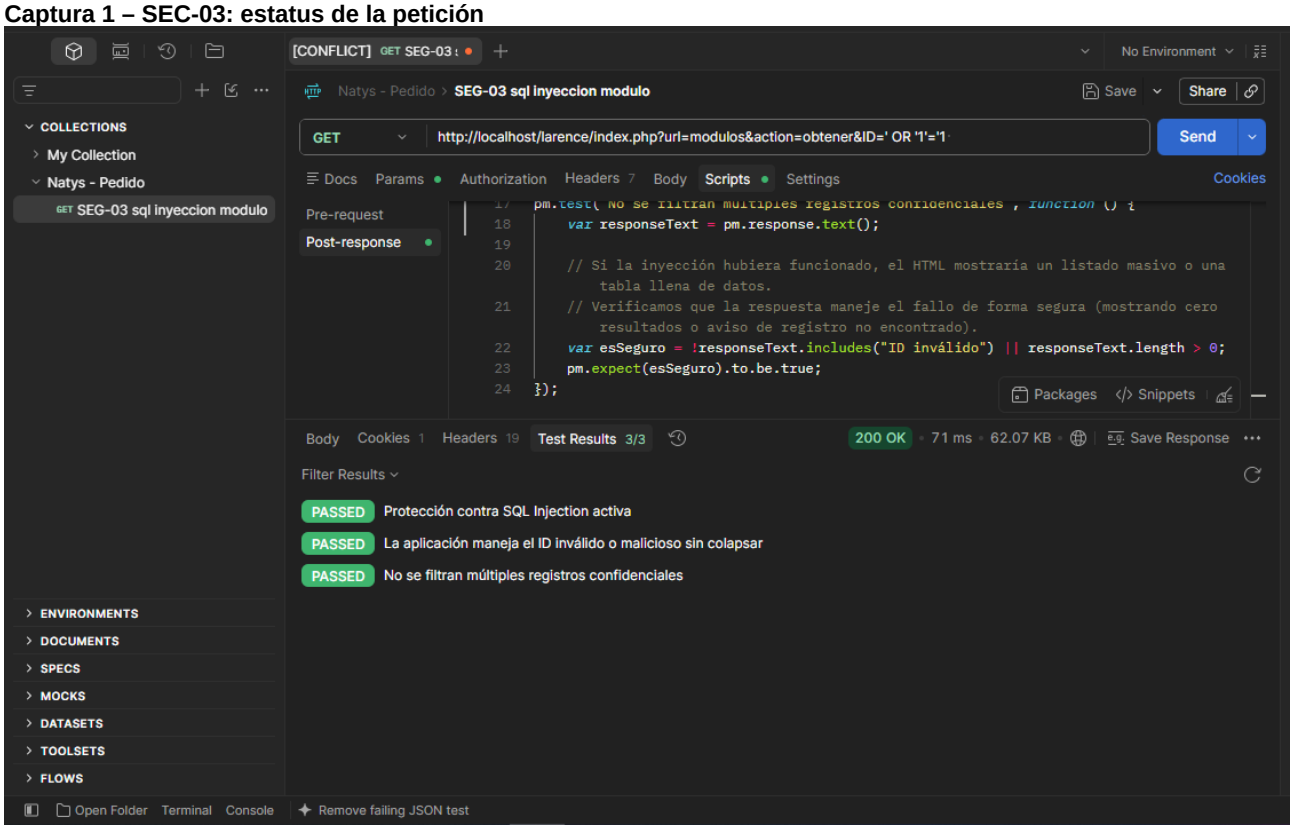
7.3 Prueba de seguridad SEC-03: inyección SQL en la gestión de módulos

La prueba envía una cadena de inyección SQL en el identificador del registro para comprobar que el sistema la trata como un valor y no como código ejecutable.

ID	SEC-03

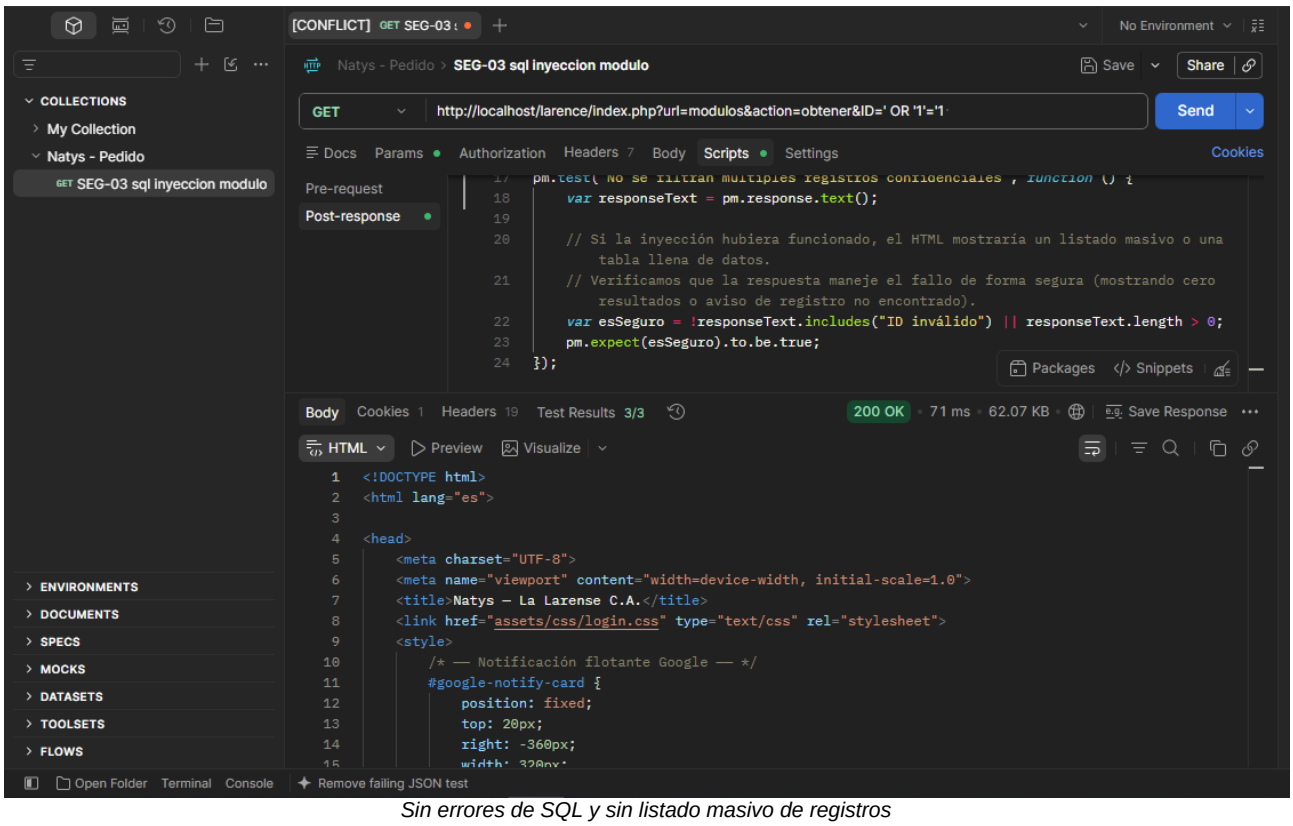
Nombre	El identificador del módulo resiste la inyección SQL
Tipo de prueba	Seguridad (inyección SQL)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=modulos&action=obtener&ID=1' OR '1'='1`
Descripción	Envía en el parámetro ID una cadena de inyección SQL clásica. Un sistema vulnerable devolvería un listado masivo de módulos o un error de SQL.
Precondiciones	Apache y MySQL activos · sesión iniciada (cookie de sesión válida en el encabezado).
Datos de entrada	`ID = 1' OR '1'='1`
Pasos	1. Se duplicó la petición de consulta y se renombró como SEC-03. 2. Se reemplazó el valor del parámetro ID por la cadena de inyección. 3. Se envió la petición GET con la sesión iniciada. 4. Se verificó que no apareciera un error SQL ni un listado masivo de registros.
Resultado esperado	Código 200, respuesta HTML, sin mensajes de error SQL y sin un listado masivo de módulos.
Resultado obtenido	200 OK en 71 ms. La respuesta HTML se procesa sin error SQL y no muestra un listado masivo de módulos. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-03: estatus de la petición



200 OK en 71 ms y respuesta HTML controlada

Captura 2 – SEC-03: respuesta del servidor (HTML)



Sin errores de SQL y sin listado masivo de registros

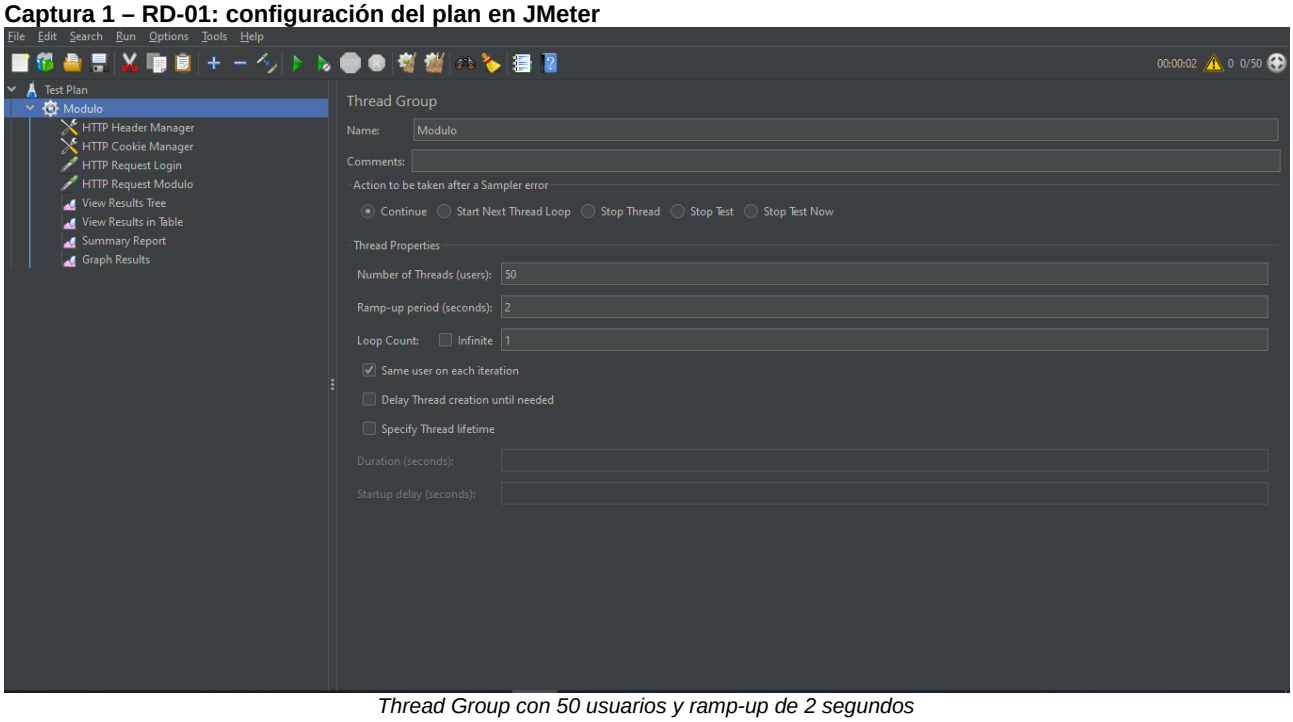
8.1 Prueba de rendimiento RD-01: Módulo

La prueba de rendimiento simula 50 usuarios virtuales que inician sesión y consultan el módulo al mismo tiempo, con el fin de medir el tiempo de respuesta y la estabilidad del sistema bajo carga (RNF 05).

ID	RD-01
Nombre	50 usuarios consultan la gestión de módulos al mismo tiempo
Tipo de prueba	Rendimiento / carga (no funcional)
Herramienta	Apache JMeter 5.6.3 · plan de prueba del módulo Módulo
Petición	`POST http://localhost/larence/index.php?url=autenticator&action=ingresar (login) + GET http://localhost/larence/index.php?url=modulos`
Configuración	Thread Group: 50 usuarios virtuales · ramp-up de 2 segundos · Loop Count infinito, detenido al completar 100 solicitudes (50 login + 50 módulo).
Precondiciones	Apache y MySQL activos · usuario de prueba válido · HTTP Cookie Manager con la cookie de sesión.
Pasos	1. Se creó el plan de prueba del módulo en Apache JMeter 5.6.3. 2. Se agregaron HTTP Request Defaults (localhost:80, ruta /larence), HTTP Cookie Manager y el Thread Group con 50 usuarios y ramp-up de 2 segundos. 3. Se agregaron las peticiones HTTP Request Login y HTTP Request Modulo. 4. Se agregó el listener Summary Report y se ejecutó el plan con el Thread Group en modo infinito. 5. Se detuvo la ejecución al completar 100 solicitudes y se registraron las métricas.
Resultado esperado	100 solicitudes procesadas, tiempo promedio menor a 2.000 ms y 0 % de errores.
Resultado obtenido	100 solicitudes (50 login + 50 consultas al módulo), promedio de 55 ms (mínimo 25 ms, máximo 111 ms), desviación estándar de 18,30 ms, 0,00 % de errores y 47,7 solicitudes/s.
Estado	**Aprobado**

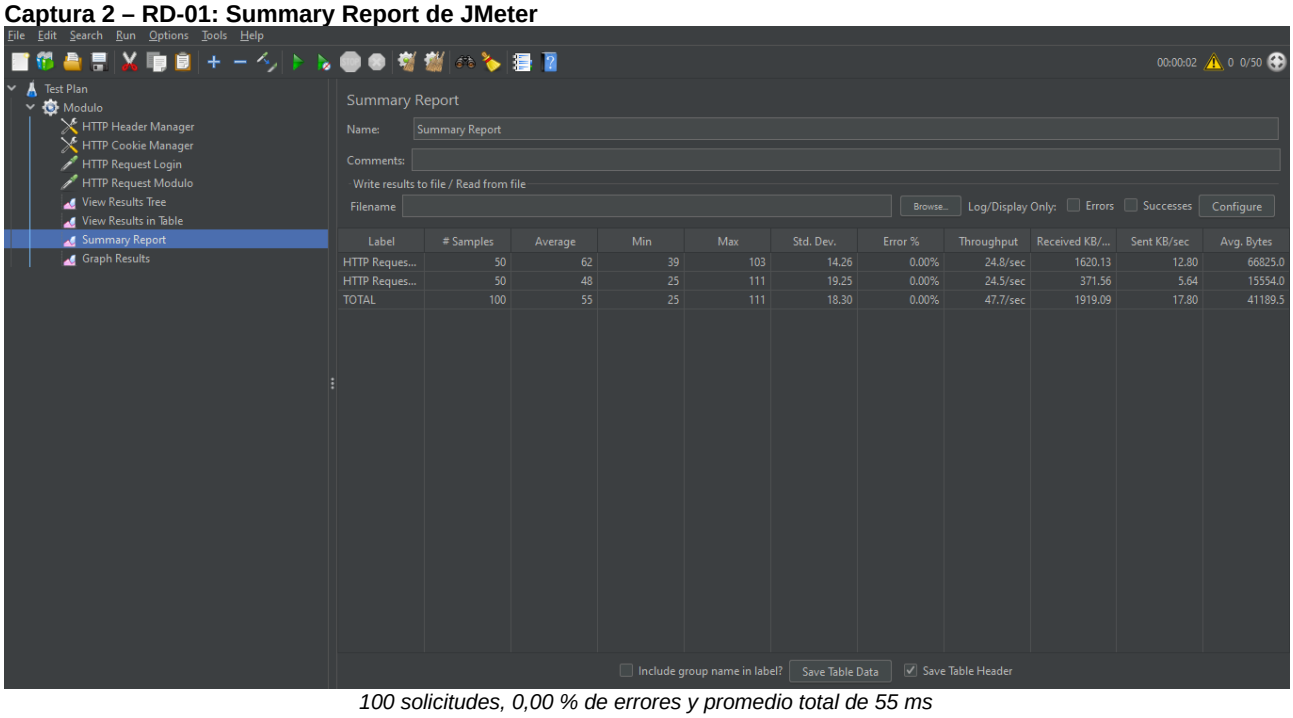
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – RD-01: configuración del plan en JMeter



Thread Group con 50 usuarios y ramp-up de 2 segundos

Captura 2 – RD-01: Summary Report de JMeter



100 solicitudes, 0,00 % de errores y promedio total de 55 ms

9.1 Prueba de aceptación AC-01: Módulo

La prueba de aceptación verifica, desde el navegador, que el módulo cumple el flujo de consulta solicitado por el cliente. Se escribió como una historia de usuario con Codeception y se ejecutó con PhpBrowser.

ID	AC-01
Nombre	El usuario consulta el listado de módulos y su gestión
Tipo de prueba	Aceptación
Herramienta	Codeception 5.3.7 con el módulo PhpBrowser
Requisitos cubiertos	CRUD Leer: consulta del listado de módulos y de su interfaz de gestión.

Archivo y método	tests/aceptacion/tests/Acceptance/ModuloCest.php · consultarGestionDeModulos
Precondiciones	Apache y MySQL activos · sesión iniciada en el sistema · cookie PHPSESSID válida.
Pasos	1. Se configuró Codeception 5.3.7 con el módulo PhpBrowser y la URL base del sistema. 2. Se fijó la cookie PHPSESSID de una sesión válida. 3. Se abrió GET /index.php?url=modulos con amOnPage y se verificó que no redirigiera a autenticator. 4. Se comprobó el texto «GESTION DE», el texto «MODULOS» y el botón #tour-btn-registrar. 5. Se ejecutó: php vendor\bin\codecept run acceptance ModuloCest -c tests\aceptacion --steps.
Resultado esperado	El listado muestra «GESTION DE» y «MODULOS», y el botón de registro #tour-btn-registrar.
Resultado obtenido	Escenario PASSED: el listado de módulos y el botón de registro se mostraron correctamente. OK (1 prueba, 4 aserciones) en 0,871 s (16 MB).
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – AC-01: ejecución de la prueba con Codeception



Escenario PASSED: el listado de módulos y el botón de registro se mostraron correctamente. OK (1 prueba, 4 aserciones) en 0,871 s (16 MB).